基于 AI 驱动的需求交付流程引擎（公开版）

AI DevFlow · 用 AI 重新定义研发流程

赛道定位与能力考察重点

本赛道考察选手对「AI Native 研发范式」的理解深度与工程落地能力。我们不是在找"会用 AI 写代码的人"，而是在找 **能让 AI 驱动整条研发流水线的人**。我们鼓励具备以下能力与背景的选手参赛：

- 是 **AI Native 的构建者**，能将 Agent 编排思维融入研发流程设计，让 AI 成为流程的主驾驶而非副驾驶。
- 具备**扎实的前后端工程能力**，能设计可扩展的流水线引擎、API 体系和 Agent 编排架构，打造极致“所见即所得”交互体验。
- 拥有**端到端交付意识**，能从需求描述出发，交付可上线、可验证的完整功能。

具体命题：DevFlow Engine · AI 驱动的研发全流程引擎

背景与痛点

传统软件研发流程是一条"人力密集型流水线"：产品经理写 PRD → 技术负责人拆解任务 → 开发者编码 → 代码评审 → 测试 → 上线。每个环节依赖不同角色、不同工具，信息在环节间反复丢失和变形。

当下的 AI 辅助开发工具（Copilot、Cursor 等）只解决了"写代码"这一个点。但真正的研发瓶颈不在敲键盘——而在**需求理解是否准确、方案设计是否合理、代码变更是否安全、测试覆盖是否充分**。

如果有一个平台，能把「需求 → 方案 → 编码 → 测试 → 评审 → 交付」整条链路编排成一条 AI 驱动的 Pipeline，每个环节由专门的 AI Agent 负责执行，人类只需在关键节点做决策确认——这就是 AI Native 研发的终极形态。

如果有一个智能助手，能让产品和设计同学直接在网页上“指点江山”，通过圈选和对话就能完成修改并立即看到预览，那么迭代效率将得到革命性的提升。

任务目标与范围

功能一：

你需要设计并实现一个 **AI 驱动的研发流程引擎（DevFlow Engine）**。它是一个平台级产品：用户输入需求描述，平台将其拆解为多阶段 Pipeline，每个阶段由 AI Agent 执行，最终产出可上线的代码变更。

🎯 **核心理念**：Pipeline 是骨架，Agent 是肌肉负责实现，人类负责监督和Review。平台定义流程的结构与编排规则，AI Agent 负责每个阶段的实际执行，人类在关键检查点做 Approve/Reject 决策。

功能二（选做，觉得功能一不够有挑战的就做）：

构建一个能够以浏览器扩展或注入脚本形式运行在目标网页上的 AI Agent。这个 Agent 需要能够完整地执行“在页面上圈选元素、通过对话修改、实时预览效果”的核心链路。

你的核心任务是打通：**页面注入悬浮控件 → 圈选元素 + 对话 → 代码修改 → 实时热更新预览 → 提交 MR** 这条自动化流水线。

💡 **核心理念：AI Agent 是主驾驶（Pilot），网页本身是你的“操作面板”**。你在页面上圈选元素并通过对话提出修改需求，Agent 负责定位和修改代码，预览页面实时反映结果，让你在同一个界面里完成“发现问题→修改→验证”的完整闭环。

研发流程参考模型

> 以下阶段是对真实研发流程的抽象，供参考。选手可自行定义阶段划分，但最终 Pipeline 必须覆盖从"需求输入"到"可交付代码"的完整链路。

阶段	输入	Agent 职责	输出
需求分析	自然语言需求描述	理解意图，澄清歧义，输出结构化需求	结构化需求文档（含验收标准）
方案设计	结构化需求 + 代码库上下文	分析现有架构，设计技术方案，确定影响范围	技术方案（含文件变更清单、API 设计）
代码生成	技术方案 + 代码库	按方案逐文件生成/修改代码	代码变更集（Diff）
测试生成	代码变更集 + 需求	生成单元测试和集成测试	测试代码 + 执行结果
代码评审	代码变更集 + 方案 + 测试结果	多维度审查（正确性、安全性、规范性）	评审报告（含问题列表和修复建议）
交付集成	评审通过的变更集	整合变更，生成最终交付物	可合并的代码变更 + 变更摘要

必须完成项 (Must-have)

功能一

1. Pipeline 引擎。设计一套可配置的流水线引擎，支持：

- 阶段（Stage）的定义、排序与依赖管理
- 每个阶段绑定一个或多个 AI Agent 执行
- 阶段间的数据流转（上一阶段的输出作为下一阶段的输入）
- Pipeline 的启动、暂停、恢复、终止等生命周期管理

2. Agent 编排与执行

- 每个流程阶段有对应的 Agent，Agent 具备明确的角色定义（System Prompt）、输入输出契约
- Agent 能感知代码库上下文（至少支持通过目录/文件路径提供上下文）
- LLM Provider 可配置（支持至少 2 个不同的模型提供商，运行时可切换）

3. Human-in-the-Loop 检查点

- Pipeline 中至少有 2 个人工检查点（如：方案设计审批、代码评审确认）
- 检查点支持 Approve（继续）/ Reject（回退到上一阶段重做，并携带 Reject 理由）
- 提供清晰的检查点 UI 或 API，展示当前阶段的产出物供人类决策

4. API-First 架构

- 所有核心操作通过 RESTful API 暴露（Pipeline CRUD、执行触发、状态查询、检查点操作）
- API 设计规范、文档完整（至少提供 Swagger/OpenAPI 或等效文档）
- 推荐 Golang 实现服务端，不做强制限制

5. 可运行的端到端演示

- 至少完成一次完整的 Pipeline 运行：从需求描述输入 → 经过全部阶段 → 产出可运行的代码变更
- Pipeline 的目标代码库可以是平台自身（即用自己的平台给自己加功能）

功能二

1. **给流水线引擎建设前端UI和官网：需自行实现 TS+React 单页应用（SPA），至少包含首页与功能页，官网风格推荐参考飞书官网的页面结构与交互风格。**
2. **注入悬浮对话框与圈选功能**：你的 Agent 需要以某种方式在你自建站点的页面内注入一个悬浮的对话框，并实现基本的元素圈选功能。
3. **支持圈选并定位修改**：用户可以通过鼠标圈选页面上**至少 3 个不同的元素**（例如一个按钮、一个标题、一段卡片文案），并在弹出的对话框中对 AI 下达修改指令。你的 Agent 需要能基于选区上下文，理解并定位到源代码中的对应位置进行修改。

4. **预览链接支持热更新**：代码修改完成后，Agent 需能触发你自建项目的热更新，让用户能立刻看到修改后的效果。
5. **自动创建 MR 并生成摘要**：在用户确认修改无误后，Agent 应能自动将本次的所有代码改动创建一个 MR，并生成一段简单的、语义化的改动摘要（说明改了什么）和行级 diff 摘要。

可选加分项 (Good-to-have)

- **多 Agent 协作**：同一阶段内多个 Agent 并行或协商工作（如：一个 Agent 写代码，另一个 Agent 同步写测试）
- **自动回归**：当评审发现问题时，Agent 自动修复并重新提交，无需人工介入（带最大重试次数控制）
- **可观测性面板**：Pipeline 运行状态的实时可视化——每个阶段的耗时、Token 消耗、Agent 的推理过程、成功/失败率
- **代码库索引**：对目标代码库进行语义索引，让 Agent 在方案设计和代码生成阶段能精准检索相关代码
- **Pipeline 模板**：支持预定义不同类型的 Pipeline 模板（如：Bug 修复流程、新功能开发流程、重构流程），用户可一键使用或自定义编辑
- **Git 集成**：自动创建分支、提交代码、发起 MR/PR

演示与验收要求

演示分为**三个环节**，逐层递进：

环节一：平台能力展示（选手自备）

功能一：选手提前准备一个需求，现场演示完整的 Pipeline 运行过程：

- 展示需求输入后，Pipeline 各阶段的自动流转
- 展示至少一个 Human-in-the-Loop 检查点的交互过程（Approve 或 Reject + 重做）
- 展示最终产出的代码变更及其质量

功能二：你的最终演示需要清晰地呈现以下流程：

- **注入与交互**：打开你自建站点的网页，展示你的 Agent 成功注入了悬浮控件。
- **圈选与修改**：演示通过鼠标圈选页面上的至少三个元素，并在对话框中通过自然语言与 AI 交互，完成对这些元素的修改（如改文案、换颜色等）。
- **实时预览**：展示 AI 完成修改后，页面实时热更新，不用刷新页面就能看到所有修改都已生效。

- **MR 交付**：展示在确认修改后，Agent 能自动创建一个 MR，并在 MR 描述中看到清晰的改动摘要。

环节二：现场命题挑战（评委出题）

评委现场给出一个功能需求（目标代码库为选手的平台自身），选手使用自己的平台完成开发并现场验证。

> 🔥 此环节核心验证：你的平台不是纸上谈兵，而是**真正能驱动研发流程跑通的生产力工具**。

环节三：技术答辩

评委针对以下维度提问：

- **架构设计**：Pipeline 引擎的设计思路、Agent 编排策略、扩展性考量
- **工程质量**：代码结构、错误处理、性能考量
- **AI Native 思维**：为什么这样设计 Agent 的角色划分？Prompt 工程的关键决策是什么？如何平衡自动化与人工干预？
- **实际价值**：这套流程相比传统研发，到底快在哪、好在哪？有什么不能做的？

通用交付物清单

- **源代码**：
  - 完整的、可独立编译运行的前后端源代码
  - 清晰的目录结构与代码注释
- **可运行程序**：
  - 提供 Docker Compose 或等效的一键部署方案，评委可在本地环境中完整运行
- **技术与项目文档**：
  - **技术方案设计文档**：详述系统架构、技术选型、Pipeline 引擎设计、Agent 编排策略
  - **安装与运行指南**：一步步指导评委如何部署、配置并运行项目
  - **测试报告**：包含功能、性能的测试用例、过程与结果
- **演示材料**